

Modularity

Table of contents

MODULARITY	1
Modularity	1
Advantages of modularity	3
Exercise - Palindrome (easy)	6
Solution	6
Exercise - Palindrome (medium)	7
Solution	8
Exercise - Palindrome (hard)	10
Solution	11
Modularity: the USE relationship	14
Relation between modules	14
Representation by oriented graphs	16
Esample - Library of Mathematical Functions	17
Cohesion and coupling	27
Example of control coupling	28
INTERFACE	30
Interaction between modules	30
Abstraction, Encapsulation, Information Hiding	31
Support to the modularity	32
EXERCISE - timeslot	32
Solution	33

MODULARITY

Modularity

One of the so-called ‘Seven principles of software engineering’

- Rigor and Formality

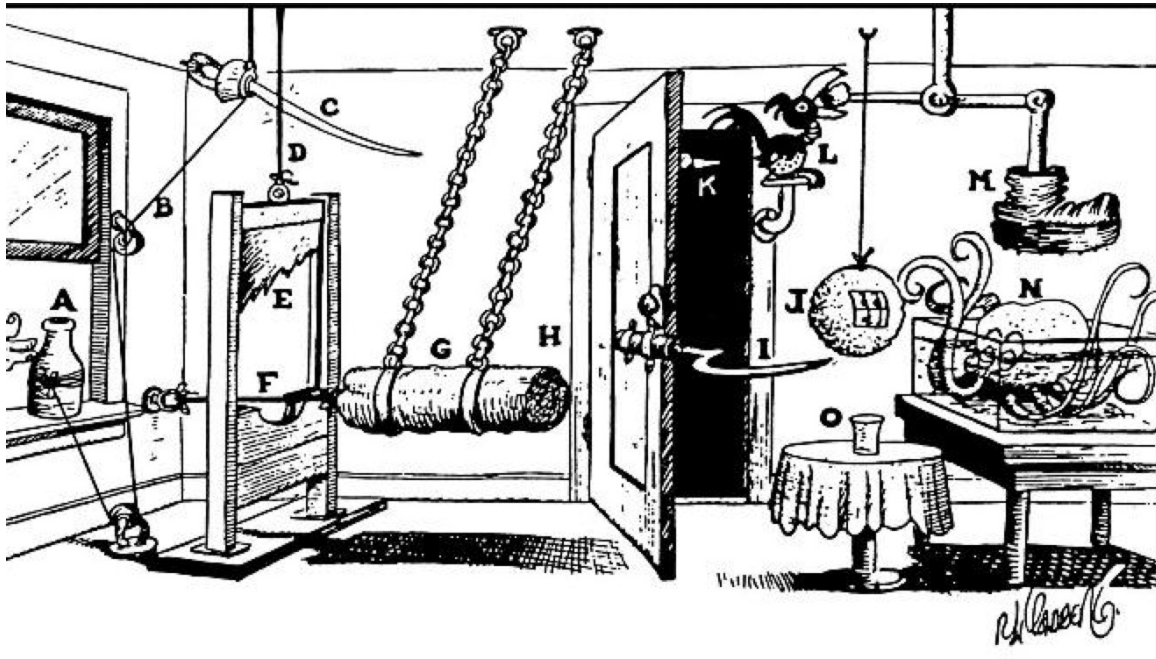
- Abstraction
 - Generality
 - Incrementality
 - **Anticipation of change**
 - **Separation of concerns**
 - **Modularity**
-

There is a limit to the complexity that a human being can handle. *We need to divide problems into more straightforward problems.* (I often see software made by one huge script...)

- A single script must be divided into **functions**.
 - The functions must be small.
 - **They must be about half of what you are thinking.**
-

Problems that arise from using a single huge script.

- We can not have everything under control.
 - It is difficult to **make changes**. How many parts of the code do we need to master before making a change? (*'non-locality'* of the code).
 - It is hard to **reuse** part of this huge script (*'non-locality'* of the code)
 - If the code does not work, we have to **check everything** to find the error.
 - It is not possible to **share** the work with other people.
 - It is hard to **understand** the code written by others. (*Or by yourself a month ago. Or maybe a week ago. Or maybe yesterday...*)
 - ...
-



A complex system that can be divided into smaller parts (modules) is called **modular**.

Our system becomes a collection of modules. The module is a 'piece' of system that **can be considered separately**.

Two different approaches exist:

- Bottom-up: first we design the modules, then we create the composition
- Top-down: first we break down problem into small pieces (subproblems), then we design a module for each subproblem

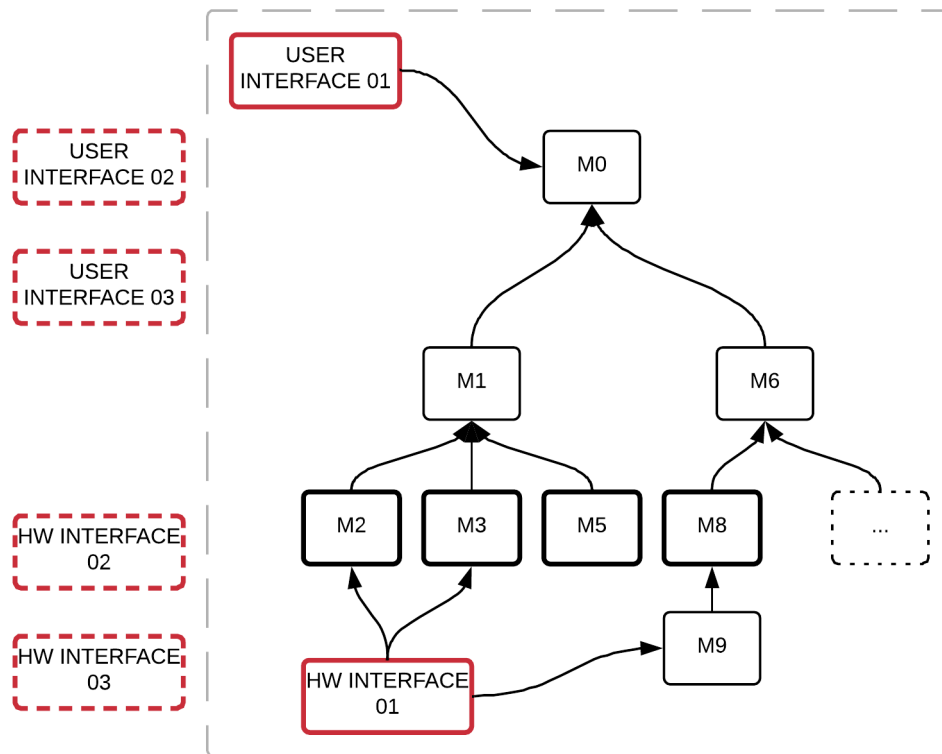
Advantages of modularity

- **manage and control complexity**
 - ability to break down a complex system into simpler parts (top-down)
 - ability to compose a complex system starting from existing modules (bottom-up)
 - face the **anticipation of change** - we can identify the modules that we will probably modify to meet future needs
 - ability to **change a system** by modifying only a small set of its parts
 - **work in groups**
-

Separation of concerns

Modularity allows us to deal with different aspects of the problem, **focusing our attention on each of them separately** (i.e. one module deals with calculating areas, another with drawing geometric figures, ...)

Example: do not use output instructions in modules that are not intended for I/O management!



- write **understandable** software: we can understand the software system as a function of its parts
 - isolate errors. Check single modules
 - **reuse** one or more modules of the software
 - ...
-

There is a limit to the complexity that a human being can handle. *We need to divide problems into more straightforward problems.*

We can **split a problem into small, easy sub-problems** (*divide et impera* - *divide and conquer*) and face with each subproblem a time.

💡 ESERCITAZIONE

Exercise - Palindrome

A *palindrome* word is a word that you can read from left to right and vice-versa, and the word doesn't change

Example: ABBA, EXE, ROTATOR

In the following exercise, let us consider only UPPERCASE characters.

To simplify the problem, we will use static allocation of memory, i.e., in a function that inverts a string, we can declare a character array in the `main`

```
void invert_string(const char *source, int n, char *dest);

int main(void){
    char dest[MAXSTRING];
    //...
```

[complete the exercise using dynamic allocation (`malloc`, etc.)]

(Don't use functions from `<string.h>`. Use only the `strlen()` function, and implement the other functions from scratch.)

- 1 - (**EASY**) Write a function that receives in input a string and returns `True` if the string is palindrome, `False` otherwise.
- 2 - (**MEDIUM**) Write a function that receives in input a string, and returns a **PALINDROME** string. The function must build the palindrome string adding characters **on the left** of the initial string. Example. Input: `'BCBAZ'`; Output: `'ZABCBBCBAZ'`
- 3 - (**HARD**) as in 2, but the function must add the **MINIMUM** number of characters. Example. input: `'BCBAZ'`; Output: `'ZABCBBAZ'`

-
- 1 - (EASY) Write a function that receives in input a string and returns **True** if the string is palindrome, **False** otherwise.

Test the function using appropriate input

Exercise - Palindrome (easy)

Example of main

```
int main(void){

    Bool answer;
    char source[MAXSTRING];
    int i;
    char *words[N_WORDS] = {"A",
                             "AA", "AX",
                             "ABA", "AAX", "AXY",
                             "ABBA", "ABAX", "AAXY", "AXYW",
                             "ABCBA", "ABBAX", "ABAXY", "AAXYW", "AXYWZ" };

    printf("\n\n");

    for(i=0; i<N_WORDS; i++){
        strcpy(source, words[i]);
        answer= palindrome(source,strlen(source));
        printf("%s PALINDROME: %s\n",source, answer ? "YES":"NO");
    }

}
```

Solution

```
//palindrome_1.c
/* Write a function that receive in input a string
 * and returns `True` if the string is palindrome, `False` otherwise.
 */

#include <stdio.h>
#include <string.h>
#define N_WORDS 15
#define MAXSTRING 100
```

```

typedef int Bool;

Bool palindrome (char *s, int n){
    int i,j;
    for (i=0; i<n/2; i++) {
        j = n - i - 1;
        if (s[i] != s[j]) return 0;
    }

    return 1;
}

int main(void){

    Bool answer;
    char source[MAXSTRING];
    int i;
    char *words[N_WORDS] = {"A",
                             "AA", "AX",
                             "ABA", "AAX", "AXY",
                             "ABBA", "ABAX", "AAXY", "AXYW",
                             "ABCBA", "ABBAX", "ABAXY", "AAXYW", "AXYWZ" };

    printf("\n\n");

    for(i=0; i<N_WORDS; i++){
        strcpy(source, words[i]);
        answer= palindrome(source,strlen(source));
        printf("%s PALINDROME: %s\n",source, answer ? "YES":"NO");
    }

    getchar();
}

```

Exercise - Palindrome (medium)

- Write a function that receives in input a string, and returns a PALINDROME string. The function must build the palindrome string adding character **on the left** of the initial string. Example. Input: **'BCBAZ'**; Output: **'ZABCBBCBAZ'**

The problem is trivial if we can use two 'helper' functions

- `invert_string` that inverts a string

- `join_string` that joins two strings
-

Example of `main`

```
// test invert_string

printf("\n\ntest invert_string\n");

for(i=0; i<N_WORDS; i++){
    strcpy(source, words[i]);
    invert_string(source, strlen(source), dest);
    printf("%s->%s\n", source, dest);
}

// test join_string

for(i=0; i<N_WORDS-1; i++){
    strcpy(source, words[i]);
    strcpy(source2, words[i+1]);
    join_string(dest, source, strlen(source), source2, strlen(source2));
    printf("%s + %s -> %s\n", source, source2, dest);
}

// test create_palindrome

for(i=0; i<N_WORDS; i++){
    strcpy(source, words[i]);
    create_palindrome(source, strlen(source), dest);
    printf("%s->%s\n", source, dest);
}
```

Solution

```
//palindrome_2.c
#include <stdio.h>
#include <string.h>

#define MAXSTRING 100
#define N_WORDS 15
```



```

void join_string(char *destination, const char* source_1, int n1, const char* source_2, int
    int i;
    for (i=0; i<n1; i++){
        destination[i]=source_1[i];
    }
    for (i=0; i<n2; i++){
        destination[n1+ i]=source_2[i];
    }
    destination[n1+n2]='\0';
};

void invert_string(const char *source, int n, char *dest){

    int i,j;
    for (i=0; i<n; i++) {
        j = n - i - 1;
        dest[i] = source[j];
    }
    dest[i]='\0';
}

void create_palindrome(const char *source, int n, char *s_palindrome){
    char s_inv[MAXSTRING];
    invert_string(source, n, s_inv);
    join_string(s_palindrome, s_inv, n ,source, n );
}

int main(void){

    char source[MAXSTRING];
    char source2[MAXSTRING];
    char dest[MAXSTRING];

    char *words[N_WORDS] = {"A",
                            "AA", "AX",
                            "ABA", "AAX", "AXY",
                            "ABBA", "ABAX", "AAXY", "AXYW",
                            "ABCBA", "ABBAX", "ABAXY", "AAXYW", "AXYWZ" };

    int i;

```

```

// test invert_string(const char *source, int n, char *dest)
printf("\n\ntest invert_string\n");

for(i=0; i<N_WORDS; i++){
    strcpy(source, words[i]);
    invert_string(source, strlen(source), dest);
    printf("%s->%s\n", source, dest);
}

// test join_string
printf("\n\ntest join_string\n");
for(i=0; i<N_WORDS-1; i++){
    strcpy(source, words[i]);
    strcpy(source2, words[i+1]);
    join_string(dest, source, strlen(source), source2, strlen(source2));
    printf("%s + %s -> %s\n", source, source2, dest);
}

// test create_palindrome
printf("\n\n test create_palindrome\n\n");

for(i=0; i<N_WORDS; i++){
    strcpy(source, words[i]);
    create_palindrome(source, strlen(source), dest);
    printf("%s->%s\n", source, dest);
}

getchar();
}

```

Exercise - Palindrome (hard)

- as in 2, but the function must add the MINIMUM number of characters. Example.
input: 'BCBAZ'; Output: 'ZABCBAZ'

HINT : split the task in subtasks:

- check if s is palindrome

- split `s` into two parts; the first part must be palindrome (QUESTION: Is it **ALWAYS** possible to get this partition?)

BCBAZ $\rightarrow$ {BCB, AZ}

- build the palindrome string.

{BCB, AZ} $\rightarrow$ ZA BCB AZ

The problem is trivial if we can use these ‘helper’ functions:

- `split_palindrome_otherstring`
- `invert_string` that inverts a string
- `join_string` that joint two strings

`split_palindrome_otherstring` is trivial if we can use

- `palindrome` (returns True if the (sub)string is palindrome)
- `copy_n_characters` copies a substring of `n` characters

Solution

```
// palindrome_3.c
#include <stdio.h>
#include <string.h>

#define MAXSTRING 100
#define N_WORDS 15

typedef int Bool;

Bool palindrome (char *s, int n){
    for (int i = 0; i<n/2; i++){
        if (s[i] != s[n-i-1]){
            return 0;
        }
    }
    return 1;
}
```

```

void copy_n_characters(char *destination, const char* source, int n){
    int i;
    for (i=0; i<n; i++){
        destination[i]=source[i];
    }
    destination[i]='\0';
};

void join_string(char *destination, const char* source_1, int n1, const char* source_2, int n2){
    int i;
    for (i=0; i<n1; i++){
        destination[i]=source_1[i];
    }
    for (i=0; i<n2; i++){
        destination[n1+ i]=source_2[i];
    }
    destination[n1+n2]='\0';
};

void split_palindrome_otherstring(const char *source, int n, char *s_palindrome, char *s_nonpalindrome){
    /* find the biggest palindrome substring [ON THE LEFT]
    * and split the string
    * i.e. 'ABCBA' -> ('ABCBA' , 'XY')
    *       'ABCDE' -> ('A' , 'BCDE')
    *       'ABCBA' -> ('ABCBA' , '')
    */
    int n1;
    for(n1=n; n1>0; n1--){
        Bool is_palindrome;
        is_palindrome = palindrome(source, n1); // check if the substring [0, i-1] is palindrome
        if (is_palindrome){
            // n1 is the length of the palindrome substring
            // source+n1 is the address of the first character of the non palindrome substring
            break;
        }
    }
    copy_n_characters(s_palindrome, source, n1);
    copy_n_characters(s_nonpalindrome, source+n1, n-n1); // from i to the end of the string
}

void invert_string(const char *source, int n, char *dest){

```

```

    int i;
    for (i=0; i<n; i++){
        dest[i]=source[n-i-1];
    }
    dest[i]='\0';
}

void create_palindrome(const char *source, int n, char *s_palindrome){
    /*
        #1. split into palindrome, other_string
        #   i.e. 'ABCBA' -> 'ABCBA' , 'XY'
        #2. add the (inverted) otherstring part of the string to the original string
        #   i.e. 'YX' + 'ABCBA'

        # PROS: easy to understand - modularity
        # CONS: slow. it is possible to optimize the function.
    */

    int n_np;
    char s_part_palindrome[MAXSTRING];
    char s_part_otherstring[MAXSTRING];
    char s_inv_otherstring[MAXSTRING];

    split_palindrome_otherstring(source, n, s_part_palindrome, s_part_otherstring);

    n_np = strlen(s_part_otherstring);
    invert_string(s_part_otherstring, n_np, s_inv_otherstring);
    join_string(s_palindrome, s_inv_otherstring, n_np, source, n );
}

int main(void){

    Bool is_palindrome;
    char dest_1[MAXSTRING];
    char dest_2[MAXSTRING];

    char *words[N_WORDS] = {"A",
                             "AA", "AX",
                             "ABA", "AAX", "AXY",
                             "ABBA", "ABAX", "AAXY", "AXYW",
                             "ABCBA", "ABBAX", "ABAXY", "AAXYW", "AXYWZ"  };

    char source[]="It matters not how strait the gate, I am the captain of my soul.";

```

```

int i, n;
n= strlen(source);

// test copy_n_characters - copy 10 charcters
printf("\n\ntest copy 10 characters\n");
printf("Source: %s\n\n",source);

for (i=0; i<n/2; i++){
    copy_n_characters(dest_1, &source[i],10) ;
    printf("from index %d->%s\n", i , dest_1);
}

// test split_palindrome_otherstring
printf("\n\ntest split_palindrome_otherstring\n");
for(i=0; i<N_WORDS; i++){
    split_palindrome_otherstring(words[i], strlen(words[i]), dest_1, dest_2);
    printf("%s->%s, %s\n", words[i], dest_1, dest_2);
}

// test create_palindrome
printf("\n\n test create_palindrome\n");

for(i=0; i<N_WORDS; i++){
    create_palindrome(words[i], strlen(words[i]), dest_1);
    printf("%s->%s\n", words[i], dest_1);
}

getchar();
}

```

Modularity: the USE relationship

Relation between modules

Let S be a software system composed of n modules.

$$S = \{M_1, M_2, \dots, M_n\}$$

A binary relation r on S is a subset of the Cartesian product $S \times S$

$\langle M_i, M_j \rangle \in r$, or $M_i r M_j$

The r relationships taken into consideration are **not reflective** (a module can not be in relation with itself)

The relation r^+ is the **transitive closure** of r on S . r^+ is also a relation on S .

We can define r^+ recursively:

considering two modules M_i and $M_j \in S$,

$M_i r^+ M_j \iff M_i r M_j$

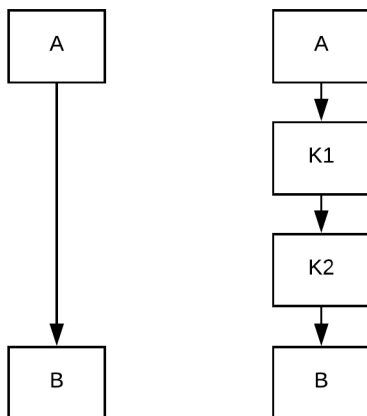
OR there is a module $M_k \in S$ st.

$M_i r M_k$ and $M_k r^+ M_j$

https://en.wikipedia.org/wiki/Transitive_closure

A **transitive closure** corresponds to the intuitive notion of **direct and indirect relation**.

$A r^+ B$ implies that $A r B$ directly OR indirectly through a chain of relations.

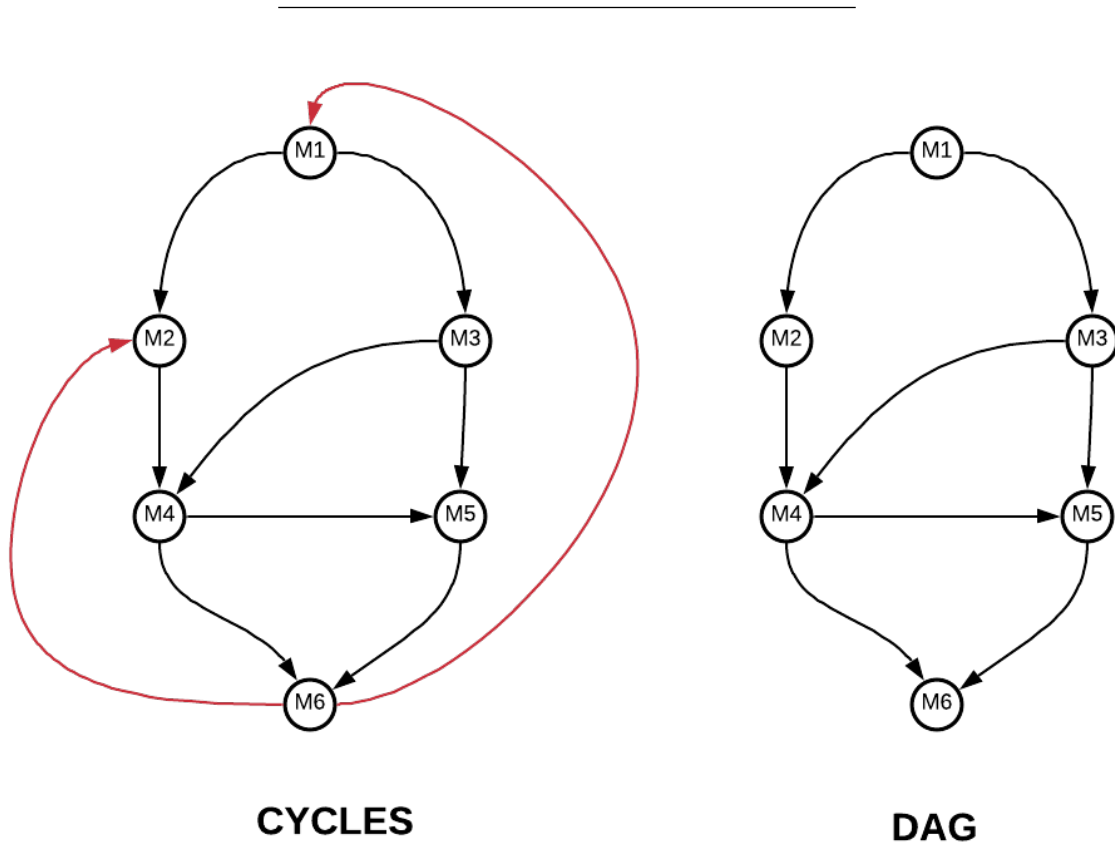


Representation by oriented graphs

We can represent the modules as the nodes of a graph.

There is a **directed arc** (or directed edge) between M_i and M_j if and only if $M_i \text{ } r \text{ } M_j$

A relation is **hierarchical** if and only if **there are no cycles** in the graph $\Rightarrow$ **DAG**,
Directed Acyclic Graph



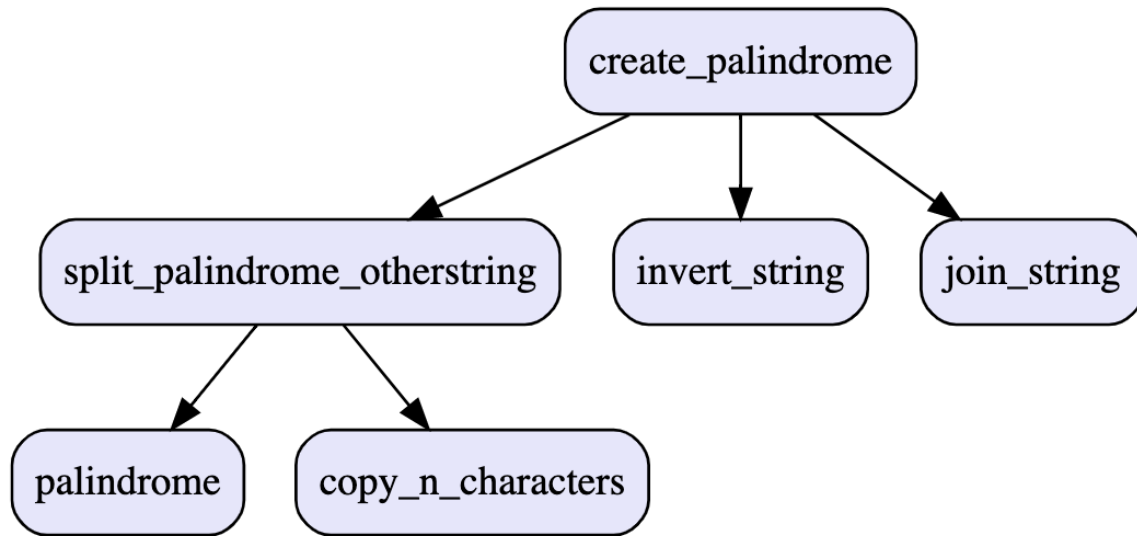
A useful relation for describing the modular structure of a software system is the so-called **USES** relation.

A USES B if A requires the presence of B to work.

A is a **client** of B. B is a **server**.

Example **A USES B** if A calls a procedure/function contained in B, or if A makes use of a type defined in B.

We can describe the solution of the *palindrome* exercise using the relation **USES**.



Example - Library of Mathematical Functions

We can describe (and design) a library of mathematical functions using the relation **USES**.

Let's focus on the trigonometric functions.

We can implement trigonometric functions using Taylor series (only):

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

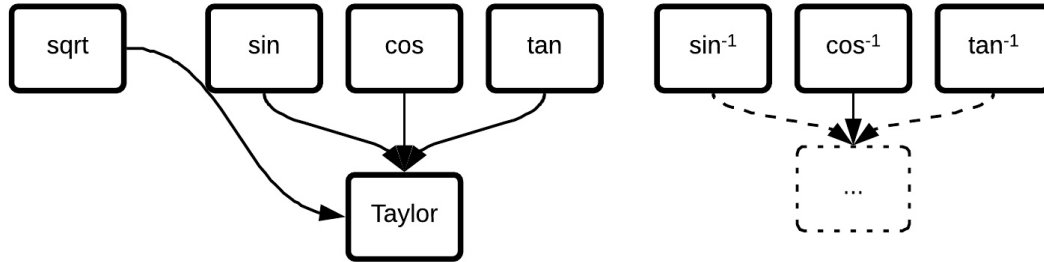
$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n}$$

$$\tan(x) = x + \frac{x^3}{3} + \frac{2}{15}x^5 + o(x^6)$$

We can implement trigonometric functions using Taylor series (only):

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}; \quad \cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n};$$

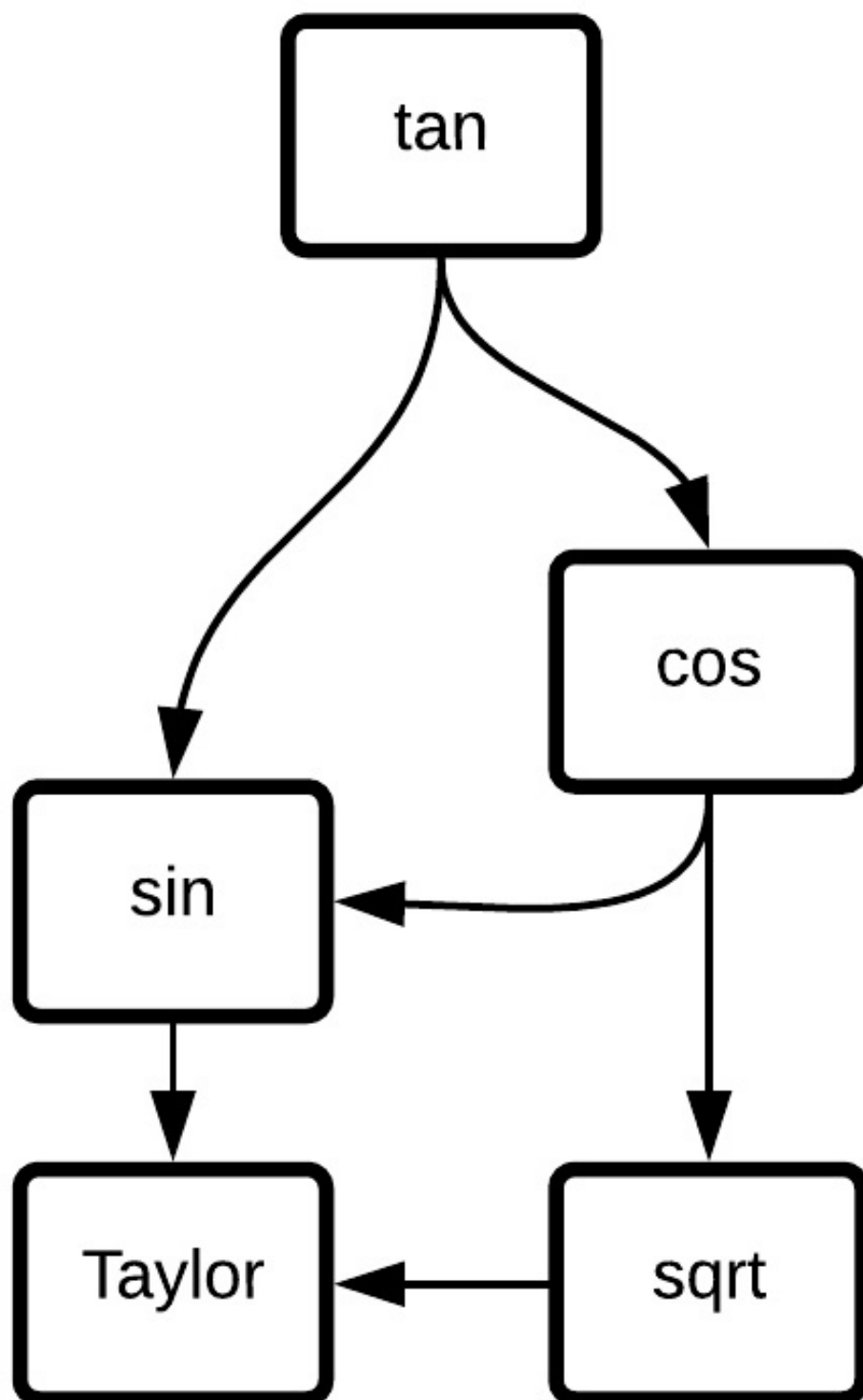
$$\tan(x) = x + \frac{x^3}{3} + \frac{2}{15}x^5 + o(x^6)$$



Or we can use (even) other relations between trigonometric functions:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}; \quad \cos(x) = \sqrt{1 - \sin^2(x)};$$

$$\tan(x) = \frac{\sin(x)}{\cos(x)};$$

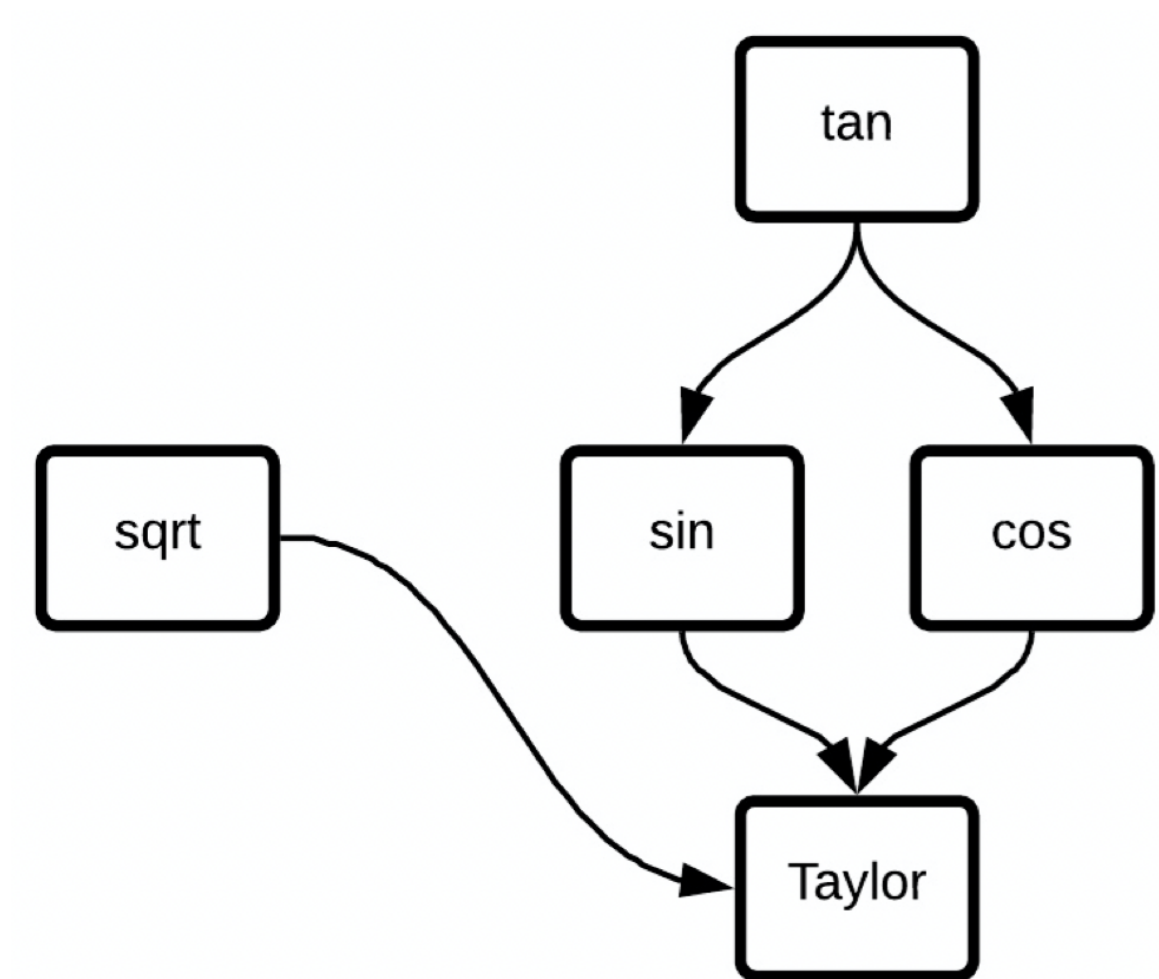


Or we can use (even) other relations between trigonometric functions:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1};$$

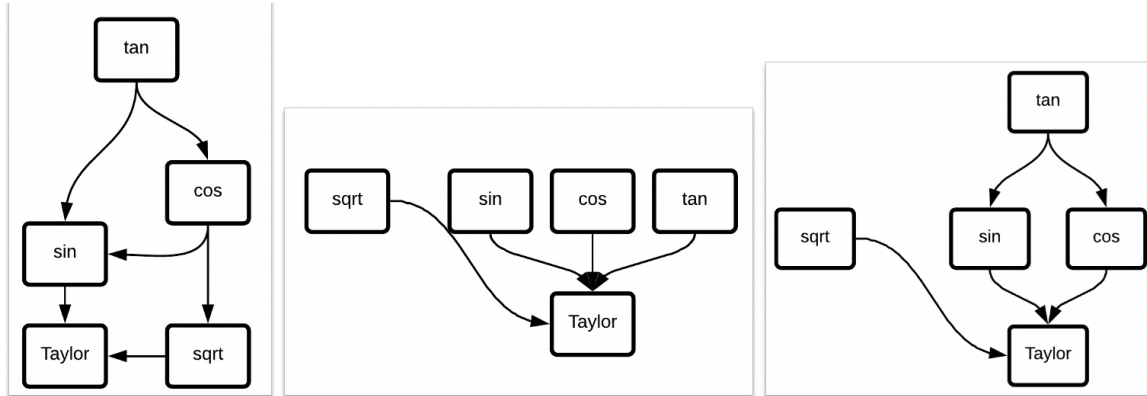
$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n};$$

$$\tan(x) = \frac{\sin(x)}{\cos(x)};$$



The **USE** relation helps us to design and describe the software system.

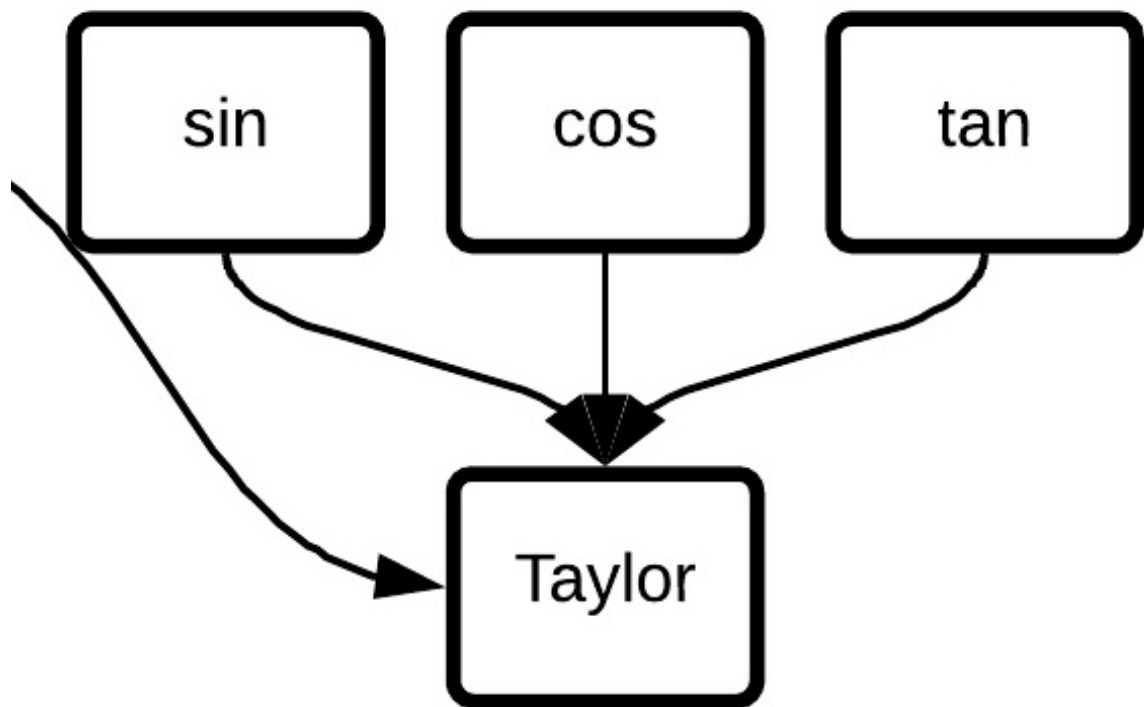
The three solutions outlined below involve three different implementations.



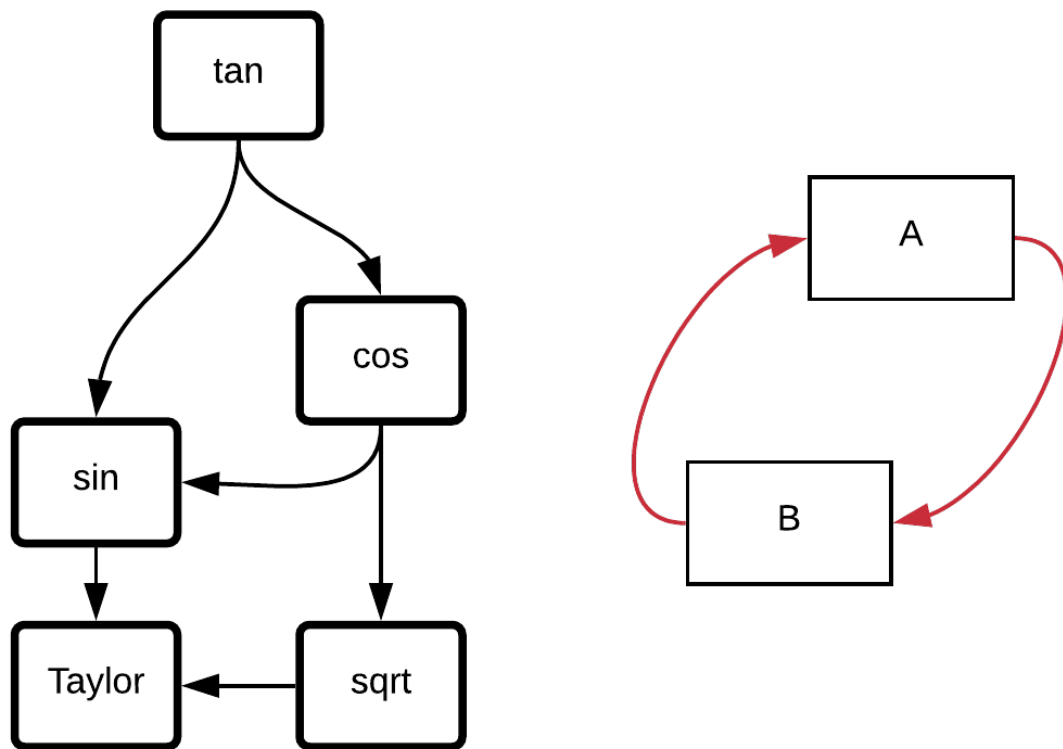
We can **impose** that the **USE** relationship is **hierarchical**.

(a generic USE relationship is not necessarily hierarchical, but it is useful to add this constraint)

- Hierarchical systems are **easier to understand** than non-hierarchical ones: once the abstractions provided by the server modules are clear, clients can be understood without having to look at server implementation.



-
- If the structure is hierarchical,
 - we can **test at least one module independently of the others** (there is at least one module that is only SERVER and not CLIENT)
 - we can test easily the **entire system**
 - If the structure is *not* hierarchical, we can have a system “where **nothing works until everything works**” [Parnas, 1979]

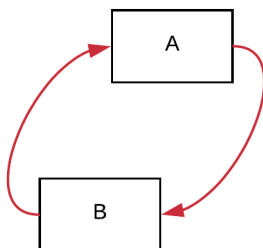


The **presence of a loop** in the **USES** relation means that **no module in the loop can be used or tested in isolation**.

For example, if

A USES B and B USES A

I need both A and B to run A or B. This configuration can also cause garbage collection problems.



```
// IT DOESN'T WORK!

typedef struct {
    int numberOfSeats;
    TypeEngine *engine;
} TypeCar;

typedef struct {
    int engineDisplacement;
    TypeCar *car;
} TypeEngine;
```

(not-so-good) SOLUTION:

```
typedef struct structCar TypeCar;
typedef struct structEng TypeEngine;

struct structCar {
    int numberOfSeats;
    TypeEngine *engine;
};

struct structEng {
    int engineDisplacement;
    TypeCar *car;
};

int main(void) {
    TypeEngine eng;
    TypeCar car;
    eng.engineDisplacement=100;
    eng.car=&car;
}
```

[see later: incomplete type in C]

Another advantage of the use of a hierarchy: **the model defines the system through abstraction levels**

The system can be seen as a series of services defined and provided by the module M_1 .

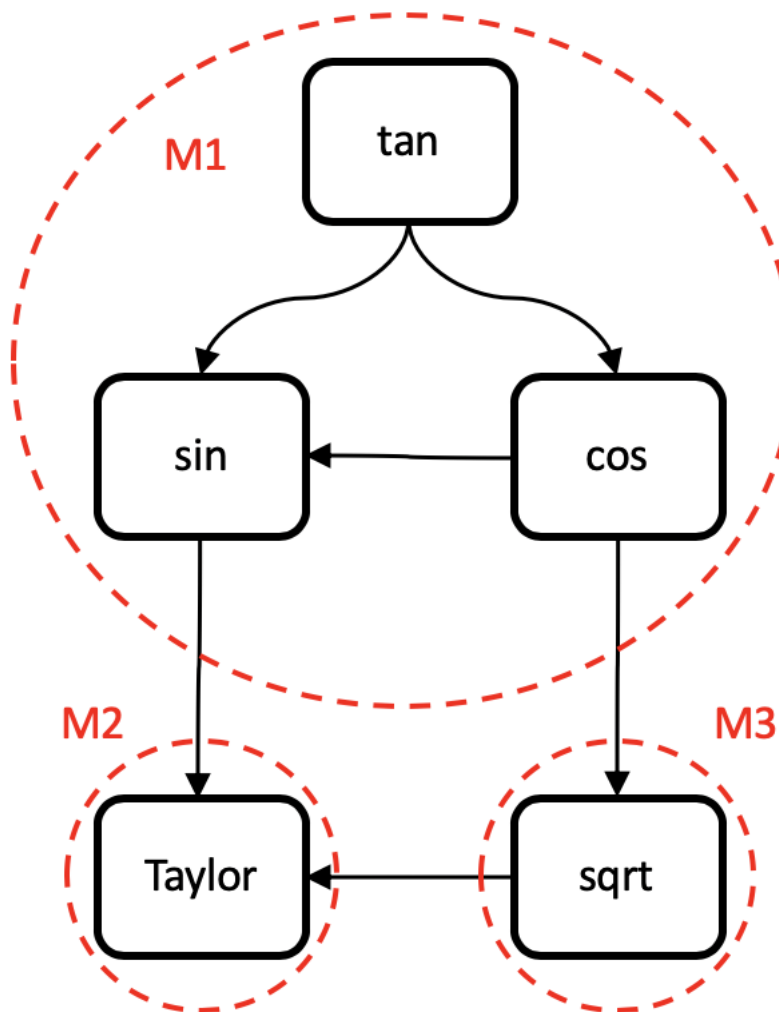
To implement such services, the module M_1 **USES** M_2 and M_3 (and so on).

The services provided by M_1 are implemented using the lower level modules.

The system can be seen as a series of services (**tan, sin, cos**) defined and provided by the module M_1 .

To implement such services, the module M_1 **USES** M_2 and M_3 .

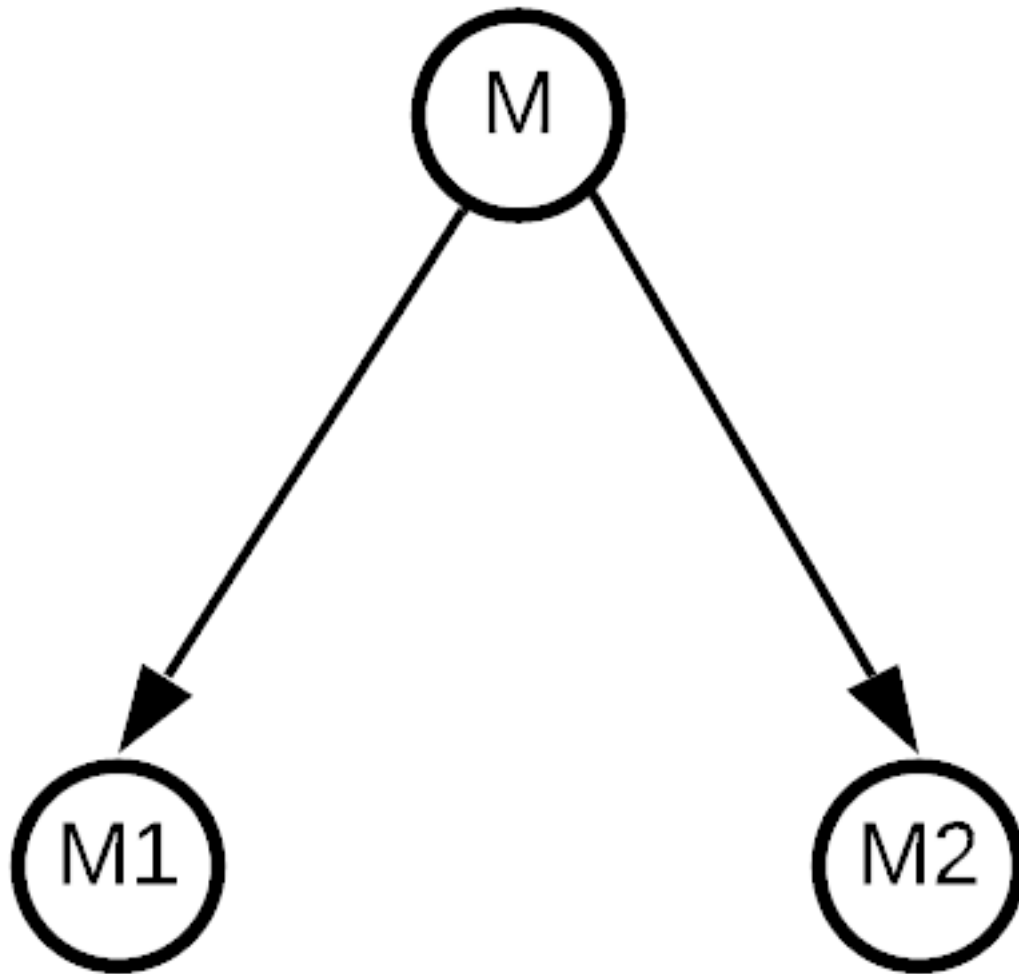
The services provided by M_1 are implemented using the lower level modules.



The **USES** relation is defined in a ‘static’ way.

Consider a M module that can call the functions $f_1 \in M_1$ and $f_2 \in M_2$. M USES M_1 and M USES M_2 , even if in a specific run only f_1 , or only f_2 , or neither is invoked.

```
if (cond1):
    if (cond2):
        f2()
    else f1()
```



Cohesion and coupling

Cohesion refers to the **degree to which the elements inside a module belong together**.

It is a measure of the strength of relationship between different parts of the same module.

Different parts of the module (instructions, functions, data,...) must be grouped for a logical reason, and must cooperate to achieve the functionality that the module must provide.

Coupling is the degree of interdependence between software modules. It measures how closely connected two modules are. Low coupling is often a sign of a good design. Two modules have a *high coupling* if they are strictly dependent on each other.

The modules must be characterized by **high cohesion and low coupling**.

Example of control coupling

```
// fizzbuzz_4.c

// Anticipation of change

#include <stdio.h>
typedef int Bool;
// don't repeat yourself!
Bool isMultiple(int n, int d){
    return n%d==0;
}
Bool isGreaterThan(int n, int d){
    return n>d;
}

void fizzBuzz(int i, int div1, int div2, int sw_f){// fizzbuzz function
    Bool fizz, buzz;
    switch (sw_f){
        case 1:
            fizz= isMultiple(i, div1);
            buzz= isMultiple(i, div2);
            break;
        case 2:
            fizz= isGreaterThan(i, div1);
            buzz= isGreaterThan(i, div2);
            break;
    }

    if (fizz && buzz )
        printf("fizzbuzz\n");
    else if (fizz)
        printf("fizz\n");
    else if (buzz)
        printf("buzz\n");
    else
```

```

    printf("%d\n",i);
}

int main(){
    int N=30;
    int i;
    int sw_f; //switch function: 1 or 2
    int div1, div2;

    // -----
    div1=3;
    div2=5;
    sw_f=1;

    printf("\n\n-----\n\n");

    for (i=1; i<=N; i++){
        fizzBuzz( i,  div1,  div2, sw_f);
    }
}

```

Key point:

the **main** module and the **fizzbuzz** module are tightly coupled

- the way the main calls **fizzbuzz** depends on an implementation detail (**sw_f == 1** calls the **isMultiple** condition, **sw_f==2** calls **isGreaterThan...**)
- changing the implementation details disrupts the functioning of the main program

Furthermore this solution has very little flexibility

- The developer responsible for the **main** program cannot define new conditions
- Inserting new conditions requires modifying the **fizzbuzz** module code by adding new branches to the conditional structure.

A possible solution consists in applying the so-called **strategy design pattern** (see later)

INTERFACE

Interaction between modules

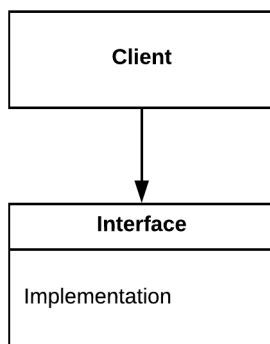
1. A module modifies the data - or even the instructions (e.g. using Assembly) - that are local to another module
2. A module can communicate with another module through a common data area, such as a global variable in C or in Python
3. A module invokes another one and transfers information using a specific *interface*. This is a traditional and disciplined way of interaction between two modules

Interface: set of services offered by the module.

The services are made available (exported) by the server module and imported by the clients.

Their implementation is a **secret of the module**.

The distinction between interface and implementation is a key aspect of good design.



The interface is a **abstraction of the module**, hiding the details that the programmer of the **clients** must not know.

- It describes the offered services, and what clients need to know to use them. The clients know the services of a module M only through its interface; **the implementation remains hidden**.
- If the interface remains the same, the module can change without causing repercussions on its clients.

- Who writes clients must know only the server interface, and can (*should*) ignore the implementation.
 - It is possible to use and test the module as a black box.
 - The code can be reused more easily.
-

What should be **shown** by the interface and what should remain **hidden** within the implementation?

The interface should reveal as *little information as possible*, but sufficient for the other modules to use the services provided.

- Revealing unnecessary information:
 - makes the interface unnecessarily complex
 - reduces the comprehensibility of the system
 - a change in implementation is more likely to be reflected on its interface (and on the clients)
-

Abstraction, Encapsulation, Information Hiding

Abstraction

The interface is an **abstraction of the module**. The knowledge of the interface is sufficient to use the module.

- It is not **necessary** to know anything else.

Encapsulation

Encapsulation is used to **hide** the details, implementation, state of a module (**the secret of a module**). The client can not know anything besides the interface. Encapsulation **prevents access** to details that are not useful.

- It is not **possible** to know anything else.

Information Hiding $\Rightarrow$ Abstraction + Encapsulation

NB: these definitions are not univocally accepted by all authors - See [Stevens, Perdita, and Rob J. Pooley. Using UML: software engineering with objects and components. Pearson Education, 2006.]

Support to the modularity

A programming language supports *modularity* and *encapsulation* to the extent that

- it automatically checks the consistency between the module and its interface
- client modules call server services based only on what is specified by the interface.

Not all programming languages fully support modularity and encapsulation.

EXERCISE - timeslot

Write a software that can **store a time interval**. The software must provide the data structure to store the time interval, and the 4 functions

- `set_h_min()`
- `set_min()`
- `print_h_min()` (or `get_h_min()`)
- `print_min()` (or `get_min()`)

The data structure can be designed in two different ways (choose only one of them)

- it can store hours and minutes
- it can store minutes

Keep in mind:

- The interface must not change if you change the implementation. *If the implementation changes, the main continues to work without changes.*
 - Hide the secret of the modules
 - ...
-

Write the data structures and functions to solve the assigned problem. They must work with this MAIN

```
// solution: timeslot
int main()
{
    struct TimeSlot ts;
    ts= set_h_min(1, 10);

    printf("TIME SLOT:\n");
    print_h_min(ts); // 1:10
    print_min(ts);   // 70

    ts= set_min(70);
    printf("TIME SLOT:\n");
    print_h_min(ts); // 1:10
    print_min(ts);   // 70
    return 0;
}
```

Solution

```
#include <stdio.h>

const int  minutesInHour=60;
// VERSION 1

typedef struct timeslot{
    int h;
    int min;
}TimeSlot;

TimeSlot set_h_min(int h, int min){
    TimeSlot ts;
    ts.h=h;
    ts.min=min;
    return ts;
}

TimeSlot set_min(int min){
    TimeSlot ts;
    ts.h = min/minutesInHour;
    ts.min = min - minutesInHour*ts.h;
```

```

        return ts;
    }

void print_h_min(TimeSlot ts){
    printf("%d : %d\n",ts.h, ts.min);
    return;
}

void print_min(TimeSlot ts){
    int min= ts.h * minutesInHour+ ts.min;
    printf("%d \n",min);
    return;
}

// VERSION 2
/*
typedef struct{
    int min;
}TimeSlot;

TimeSlot set_h_min(int h, int min){

    TimeSlot ts;
    ts.min = min + h*minutesInHour;
    return ts;
}

TimeSlot set_min(int min){
    TimeSlot ts;
    ts.min=min;
    return ts;
}

void print_min(TimeSlot ts){
    printf("%d \n", ts.min);
    return;
}

void print_h_min(TimeSlot ts){
    int h, min;

```

```

    h=ts.min/minutesInHour;
    min=ts.min-minutesInHour*h;

    printf("%d : %d\n",h, min);
    return;
}

*/
int main()
{
    TimeSlot ts;
    ts= set_h_min(1, 10);

    printf("TIME SLOT:\n");
    print_h_min(ts);
    print_min(ts);

    ts= set_min(70);
    printf("TIME SLOT:\n");
    print_h_min(ts);
    print_min(ts);

    getchar();
    return 0;
}

```

REMARKS

This solution highlights the **abstraction** and the role of the interface. If the implementation changes, MAIN can manipulate the data using the same interface.

However, the data is **NOT encapsulated**. The MAIN could directly access the fields of the structure.
